

CARMA: Online Cluster-Adaptive Eviction for GPTCache

Cluster-Adaptive Reuse and Miss-pressure Admission

Talya Yermiyahu — Final Project Report, September 2026

Executive summary. CARMA is an optional, bounded, CPU-only eviction policy that groups requests online while preserving GPTCache’s default behavior. In ten paired 10,000-request category-shift runs at capacity 100, its valid-hit rate was 4.885 percentage points above the better LRU/LFU baseline (95% CI [4.677, 5.088], Holm-adjusted $p = 0.0234$). The pollution-return difference was only +0.0067 points (95% CI [0, 0.0167]) and therefore inconclusive. Full-path latency, throughput, and memory were similar across policies. Question-pair calibration did not meet the predeclared rule for opening the final test set, so the report makes no unseen-data semantic-accuracy claim.

Software baseline	GPTCache c59fb3a6152a4458b2a070ca183b61c4b614095f
Verified revision	bea5c64ebfac5a00632d8c3db046518d668c42a8
Primary configuration	topic .70, cell .97, half-life 500, quota strength 1.0

1 Introduction and related work

A semantic cache saves an earlier answer and may reuse it when a later question has a similar numerical representation. That representation is called an *embedding*: a vector that places similar text near similar text. GPTCache is a modular open-source semantic cache. Its embedding model, similarity search, similarity check, storage, and eviction policy can be configured separately [1]. An eviction policy chooses which stored entry to remove when the cache is full.

GPTCache was selected because it is a mature, locally runnable baseline with a published system paper, an established test suite, and separable embedding, search, similarity, storage, and eviction components. Its SQLite–FAISS path runs on a central processing unit without a paid model call or application programming interface key. That modular path made it possible to change the eviction policy while preserving the existing cache interface and testing the unchanged LRU and LFU paths. The pinned baseline commit, rather than a moving default branch, fixes the interface and behavior used for every comparison.

At the software version used here, GPTCache offers global least recently used (LRU), least frequently used (LFU), first in, first out (FIFO), and random eviction. These policies use age or access count. They do not know whether several stored entries cover almost the same meaning, or whether recent misses show growing demand for a new subject.

SCALM was the main conceptual starting point for CARMA. It showed that semantic structure can guide cache management, not only decide whether two requests are similar. SCALM studies two collections of conversations, MOSS and LMSYS, finds semantic patterns at several levels, and ranks those patterns using evidence such as cache hits and saved language-model tokens. It reports large relative gains over GPTCache baselines [2].

I used that result to make meaning part of CARMA’s eviction decisions. CARMA groups stored requests into broad topics and narrower reuse cells, then uses those groups when allocating cache space and choosing what to keep. CARMA does not reproduce SCALM’s method. SCALM discovers and ranks patterns offline from a prepared collection of data; CARMA learns a bounded set of groups online while requests arrive. CARMA also adds time-decayed demand, admission based on evidence of repeated demand, topic quotas, and failure-safe integration with GPTCache. It does not require token costs, offline ranking, a separately trained model, or a paid call to a large language model. This project therefore asks a different question:

Can a policy learn useful groups of similar requests *while requests are arriving*, without an earlier ranking step, a separately trained model, token costs, or a paid call to a large language model?

Earlier cache research provides useful ideas. The 2Q policy keeps a short history so that a single unusual request does not push out useful entries [3]. Least Recently/Frequently Used (LRFU) combines how recent and how frequent an access was [4]. Adaptive Replacement Cache (ARC) changes its balance between recent and frequent entries [5]. TinyLFU estimates frequency before admitting a new entry [6]. CARMA also combines recent and repeated demand, but its main difference is that it organizes entries by meaning while requests arrive and controls how much space each topic receives. The project does not claim that the earlier cache ideas are new.

This question creates three engineering requirements. First, CARMA may decide what to keep, but it may not decide that an answer is correct. GPTCache’s normal similarity threshold still controls whether an answer can be reused. Second, CARMA’s internal data must have a fixed upper bound based on cache capacity, rather than growing with the number of requests. Third, eviction must remove both copies of an entry: the ordinary database record and the vector used for similarity search. Otherwise, an old vector in the Facebook AI Similarity Search (FAISS) index could appear as the closest result and hide a valid result when only the single closest neighbor is requested.

Contribution and evidence boundary

The contribution is CARMA: a deterministic policy that learns while requests arrive, together with backward-compatible GPTCache integration, safe cleanup after failures, state reconstruction after restart, tests, and a reproducible evaluation process. The clearest evidence that CARMA adapts to change is the +4.885-percentage-point valid-hit improvement in the changing-category workload; its paired confidence interval is entirely above zero. The full-path experiment compares LRU, LFU, and CARMA in five new processes per policy using real text. It describes request time, throughput, memory, and cache hits on one computer. The Quora Question Pairs study stops after calibration, so it does not support a broader claim about semantic accuracy on unseen data.

2 How CARMA works

2.1 Stored state and grouping while requests arrive

Each entry accepted into the cache is assigned to two semantic groups. A *topic* is a broad group; a *cell* is a narrower group of very similar requests inside a topic. The entry records when it was inserted, when it was last reused, and a hit score that becomes smaller over time. A topic records its center, its entries, recent demand, and recent misses. A cell records its center, entries, evidence of repeated demand, and most recent observation. A center is the average direction of the vectors in a group. For a cache of capacity C , CARMA keeps at most $\min(64, C)$ topics and $2C$ cells.

When no cached answer is found, the request vector joins the closest topic if its cosine similarity is above `topic_threshold`; otherwise CARMA creates a topic, within the fixed limit. It assigns a cell in the same way using the higher `cell_threshold`. Group centers are updated gradually with a normalized exponential moving average, which gives more weight to recent observations. If two choices are equal, CARMA always uses the lower stable identifier. An empty cell with little evidence can remain briefly as bounded “ghost” history. A second similar miss can then show repeated demand even if the first candidate was not admitted.

CARMA advances one logical time step after each cache hit or inserted miss. A score x last updated at time t_0 is reduced only when it is needed:

$$x(t) = x(t_0) 2^{-(t-t_0)/H},$$

Here, H is `demand_half_life`, the number of time steps needed for a score to fall by half. Setting $H = \infty$ turns decay off. Computing decay only when needed keeps the usual cache-hit path constant-time.

2.2 Space allocation, entry value, and admission

For topic c , CARMA calculates recent miss pressure p_c and quota weight w_c as follows:

$$p_c = \frac{m_c + 1}{d_c + 2}, \quad w_c = \max\{\epsilon, d_c p_c\}^q,$$

Here, m_c is the recent miss score, d_c is recent demand, and q is `quota_strength`, which controls how strongly topic demand affects space allocation. CARMA converts the weights into whole-number quotas, one for each topic, whose sum is exactly the cache capacity. It uses the largest-remainder method: assign the whole-number parts first, then give the remaining slots to the largest fractional remainders. A stored entry e in cell s has the following retention value:

$$V_e = \frac{\text{support}(s)}{\max(1, |s|)} + 0.25 \text{ hitmass}(e).$$

The first term rewards a cell that has repeated evidence and divides that reward among entries with nearly the same meaning. The second term rewards an entry that has been reused.

Before the cache is full, every valid candidate is accepted. When the cache is full, CARMA rejects a candidate if its cell has less than 1.5 units of decayed repeat evidence or if its topic has no allocated space. Otherwise the candidate competes with the weakest stored entry in its own topic or in the topic furthest above its quota. The candidate must be worth at least 1.05 times the entry it would replace. Ties are broken, in order, by value, time of last hit, insertion age, and stable identifier.

2.3 Computing cost and intended trade-off

A cache hit takes constant time, written $O(1)$, except when quotas are occasionally refreshed. Assigning a miss compares its D -dimensional vector with at most T topics and S cells, so its cost is $O((T + S)D)$. Choosing an entry to remove can inspect the C stored entries, so its cost is $O(C)$. CARMA therefore spends more processor time on policy decisions than LRU or LFU. The intended benefit is better coverage when popular subjects change or when a scan introduces many one-time requests. A simple baseline may still be as good or better when it already keeps the useful working set.

3 Software safety and GPTCaché integration

In this section, *correctness* means that the software and its stored records remain consistent. It does not mean that a reused answer is factually correct for two similar questions. Section 7.3 explains the separate duplicate-question test and why accuracy on the final test set is unknown.

CARMA is optional. If it is not selected, the factory default and public interface behave as before for valid inputs. The shared code that normalizes vectors, checks input, and cleans up storage was strengthened. A user selects CARMA with `policy="CARMA"`. The factory then connects CARMA to the storage-cleanup function itself. This prevents a custom eviction object from forgetting to remove an evicted record from external storage. New entries pass their normalized vectors and saved metadata through a backward-compatible extension point.

Property that must always hold	How it is enforced and tested
Bounded counts and state	Topic quotas sum to capacity C . Random tests check that the cache never exceeds capacity. Topics, cells, ghost history, and custom-identifier counters have fixed limits. If a required registry is full, the operation stops safely.
Database/vector agreement	An evicted entry is removed from the FAISS vector index before the ordinary database change is committed. Targeted rollback keeps earlier deletion markers and stops safely if the two stores cannot be made consistent.
One cleanup call	Duplicate victim identifiers are removed from a batch, and entries that remain in the cache are filtered out before cleanup is called outside the lock.
Invalid metadata	Missing vectors, infinite or not-a-number values, empty vectors, and vectors with the wrong number of dimensions are rejected safely when the cache is full.
Restart	Existing rows are rebuilt by reading stored vectors without changing them. Recent-demand and hit scores intentionally restart at zero and are learned again.
Repeatable decisions	Every tie between topics, cells, quotas, donor topics, victims, or ghost records has a documented stable ordering.

Table 1: Main safety properties covered by focused tests, randomized tests, failure tests, and integration tests using SQLite and FAISS.

Failure tests cover cleanup functions that raise errors, identifiers that cannot be hashed or have the same printed form, removal and readmission in one batch, vector-deletion failure after a reversible database deletion, and reconstruction after closing and reopening storage. If cleanup cannot recover, the manager marks itself unhealthy and stops instead of continuing with different internal and external records.

3.1 Layers of testing

Validation has four layers: (1) focused CARMA unit tests and randomized property tests; (2) relevant existing GPTCaché tests for eviction, SQLite, and FAISS; (3) fixed continuous-integration request traces with expected

file formats and file hashes; and (4) real SQLite and FAISS runs in new child processes. Python 3.12.13 is the main fixed version. A separate Python 3.8 test protects syntax supported by older GPTCache installations. Optional storage systems are not allowed to install packages during verification.

At verified repository commit `bea5c64ebfac5a00632d8c3db046518d668c42a8`, the public continuous-integration run passed all four jobs. The Python 3.8 slice passed 44 tests. Host verification passed eight upstream eviction tests, two SQLite–FAISS tests (with one unavailable optional case deselected), and 1,292 project tests with four platform-dependent skips, followed by the deterministic benchmark. Two fresh network-isolated containers repeated the same verification and produced matching non-timing benchmark evidence. The exact run is linked in the Artifact Map; these are retained results, not tests rerun for this report revision.

3.2 What is lost after restart

After a restart, CARMA rebuilds topics and cells from the vectors already in storage. It does not save the recent-demand and hit scores; those scores begin at zero and are learned again from new traffic. Saving them correctly would need a versioned metadata format and migration rules for every GPTCache storage system. That work is left for the future rather than pretending that old adaptive state was preserved.

4 How the experiments were run

The experimental rules were fixed before the final statistical test. A *seed* is a number that makes a generated request trace repeatable. Validation seeds, used to choose settings, were different from test seeds, used to report final results. Test results were never used to choose settings. The search evaluated 96 combinations: topic thresholds $\{.70, .80, .90\}$, cell thresholds $\{.95, .97\}$, demand half-lives $\{100, 500, 2000, \infty\}$, and quota strengths $\{0, .5, 1, 2\}$. Three validation seeds across three workloads selected the setting with the best valid-hit rate after normalizing across workloads, provided that it produced no false synthetic hits. The selected setting was $(.70, .97, 500, 1.0)$.

Workload	Fixed request pattern	What it tests
Stationary Zipf	10,000 requests; a few items are popular; 15% appear once	Stable unequal demand and protection against worse results.
Changing categories	Five phases of 2,000 requests; 80% of demand moves	How quickly the cache follows newly popular topics.
Pollution scan	3,000 warm-up requests, 4,000 unique requests, then 3,000 return requests	Resistance to one-time entries and recovery of the useful working set.
MOSS conversations, novel-long	200 unique requests from the longest quarter, balanced by token-count ranges	A known all-miss check for exact-key LRU and verification of recorded answers and token counts; not a CARMA performance comparison.
Quora Question Pairs calibration	Calibration and final-test sets share no connected question components	The trade-off when classifying duplicate questions; it does not prove that one answer is correct for both.

Table 2: Main and diagnostic workloads. The main synthetic tests use a cache capacity of 100 entries and ten paired seeds. Real-system tests use five new processes per policy, or 15 processes in total.

A cache hit is *valid* only when the returned answer identifier is allowed for the request’s known simulated concept. A false hit is a reused answer from the wrong concept and counts as saving no valid tokens. The main measurements are the percentage of all requests with valid hits, the percentage of hits that are valid, the percentage of all requests with false hits, the percentage of possible hits that were found, and a synthetic weighted estimate of savings. The synthetic weight for concept (t, c, q) is the fixed formula $16 + (31t + 17c + 13q) \bmod 241$. It is only a test weight; it is not a real token count and is not a SCALM measurement. The MOSS experiment records real corpus tokens separately. System measurements include request-time percentiles, requests per second, processor time, sampled physical memory, and checks that the two storage systems remain consistent.

Each policy is compared with another policy on the same seed and request trace. Confidence intervals are estimated by resampling the ten paired seed differences 10,000 times. The final valid-hit comparisons against LRU, LFU, and the better baseline in each seed use exact Wilcoxon signed-rank tests. The Holm correction

adjusts their p -values because several related comparisons were tested. A 95% confidence interval summarizes uncertainty caused by differences between seeds; an interval entirely above zero supports a positive average difference. A p -value measures how surprising the paired differences would be if there were no systematic policy difference. The predeclared level for these tests was 0.05. Additional, exploratory tests reuse fixed traces at capacities 20, 50, and 200; the main test uses capacity 100. *Ablation* tests turn off or replace one mechanism at a time: broad-topic grouping, decay, admission, or quota-aware eviction. As Section 6 explains, the broad-topic and quota variants are not an orthogonal pair and therefore do not identify two separate effects.

The synthetic vectors were designed with known cosine similarities. Repeated requests for the same concept have similarity 1.0. A deliberately different near-twin has similarity 0.96. Other different concepts from the same source cell have similarity 0.88, and concepts in different cells of one topic have similarity 0.72. Similarity between topics ranges from 0.00 to 0.12, below the topic threshold of 0.70. At the selected cell threshold of 0.97, two different synthetic concepts never enter the same learned cell. Therefore the main synthetic improvement does not demonstrate that CARMA groups natural-language paraphrases. It mainly tests time decay, admission, and retention under a controlled vector layout.

5 Synthetic results

A *percentage-point* difference is the direct subtraction of two percentages. For example, 68.261% minus 63.376% is 4.885 percentage points.

Workload	LRU (%)	LFU (%)	CARMA (%)	Difference (pp)
Stationary	43.457%	47.514%	54.636%	+7.122 pp
Category shift	63.376%	52.181%	68.261%	+4.885 pp
Pollution scan (whole)	58.400%	59.198%	59.200%	≈0 pp (+0.002)
Pollution return phase	97.333%	99.993%	100.000%	+0.0067 pp

Table 3: Average valid-hit rates across ten seeds at cache capacity 100 entries. For each seed and measurement, the better baseline is selected before the paired difference is calculated. “pp” means percentage points.

In the changing-category workload, CARMA’s valid-hit rate was 4.885 percentage points higher than the better baseline. The 95% confidence interval was [4.677, 5.088], the Holm-adjusted p -value was 0.0234, and CARMA beat the comparison policy in every paired run (rank-biserial effect size 1.0). In the stationary workload, the improvement was 7.122 percentage points, with a 95% confidence interval of [6.572, 7.709] and the same adjusted p -value. Relative to the stronger baseline mean, these are improvements of 7.71% for changing categories and 14.99% for stationary traffic. Percentage-point differences remain the primary effect measure because they do not inflate changes near a small denominator. For the synthetic weighted-savings estimate, the differences were +9.503 percentage points [8.641, 10.443] for stationary traffic, +5.251 [5.009, 5.475] for changing categories, and +0.0015 [0, 0.0037] for pollution. These savings use the fixed test weights described above, not observed token counts.

During the return phase after the pollution scan, CARMA’s valid-hit rate was only 0.0067 percentage points higher than the better baseline. The 95% confidence interval was [0, 0.0167] and touches zero; the adjusted p -value was 1.0. This is a neutral result, not evidence that CARMA is better. CARMA returned valid answers for 30,000/30,000 requests, while LFU returned 29,998/30,000. The full comparison is stored in `artifacts/samples/analysis/scan-return.svg`. In a separate exploratory test at similarity threshold 0.95, each policy made one deliberately induced false hit per 200 requests, a 0.5% false-hit rate. This test checks measurement logic; it was not used to choose settings and is not part of the main 0.97 operating point.

Figure 1 shows the paired main results and how the changing-category result varies with cache capacity. The pollution result is shown near zero and should not be read as a CARMA advantage.

Synthetic effects and capacity sensitivity

Retained fixed-trace CSV evidence; valid-hit rate

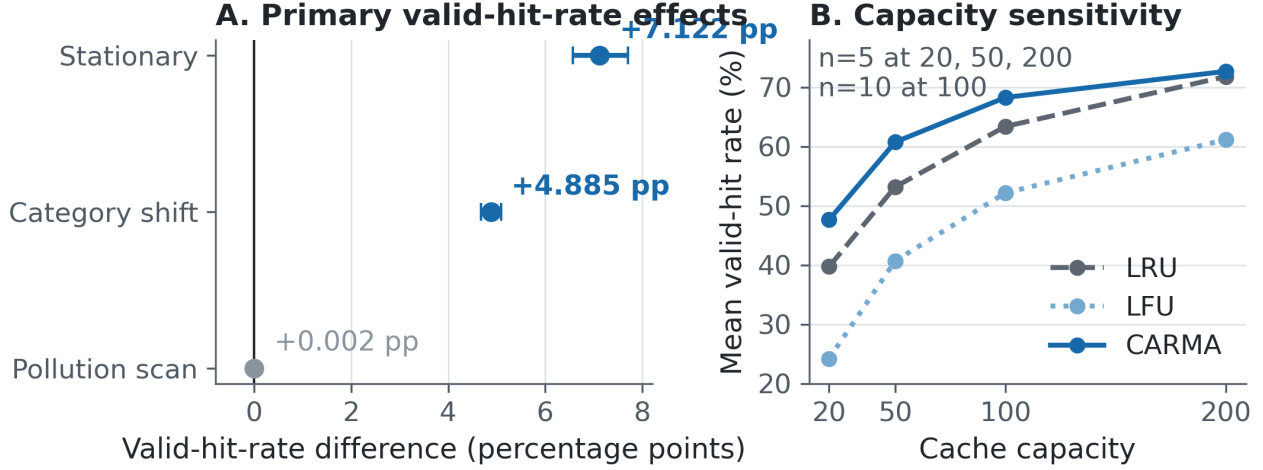


Figure 1: Synthetic results calculated from saved comma-separated-value files. Panel A shows CARMA’s valid-hit rate minus the better LRU/LFU result for each seed over the whole request trace. Bars show paired 95% bootstrap confidence intervals; the zero line means no difference. The pollution difference is +0.002 percentage points, effectively zero. Panel B shows the average valid-hit rate for changing categories at each capacity. Capacities 20, 50, and 200 use five exploratory seeds; capacity 100 uses the ten main seeds.

6 Sensitivity and component-removal tests

For changing categories, CARMA’s average valid-hit rates were 47.636%, 60.740%, 68.261%, and 72.676% at capacities 20, 50, 100, and 200. The corresponding better-baseline averages were 39.740%, 53.192%, 63.376%, and 71.792%. The differences were therefore +7.896, +7.548, +4.885, and +0.884 percentage points. Capacities 20, 50, and 200 used only five exploratory seeds; capacity 100 used the ten main seeds. The curve is useful for checking sensitivity to capacity, but the four points do not form one shared statistical test.

The retained 96-configuration validation grid also supports a descriptive parameter-sensitivity check. Table 4 averages the normalized validation score over all settings of the other parameters. It is not a causal comparison, but it shows that the selected configuration was not reported without context and that it ranked first in the complete grid.

Parameter	Aggregated normalized validation means	Best level
Topic threshold	.70: .9839; .80: .9707; .90: .9423	.70
Cell threshold	.95: .9656; .97: .9657	.97
Demand half-life	100: .9649; 500: .9695; 2000: .9662; ∞: .9620	500
Quota strength	0: .9525; .5: .9675; 1: .9723; 2: .9704	1

Table 4: Descriptive sensitivity calculated from the retained validation grid. Each mean aggregates over the other three parameters and the three validation workloads. The selected (.70, .97, 500, 1) configuration had the highest individual normalized score, .9937, among all 96 configurations.

Time decay is the clearest source of adaptation: turning it off reduced the changing-category valid-hit rate by 8.281 percentage points, although it helped stationary traffic slightly. Admission matters most during the pollution scan; turning it off reduced valid-hit rate by 3.812 points. Replacing broad topic groups with one global topic changed stationary performance by +0.347 points and changing-category performance by -0.387 points, while pollution was unchanged; narrow cell assignment was still active. Turning off quota-aware eviction produced exactly the same numbers as using one global topic, even though the number of topics differed. This experiment therefore cannot separate the effect of broad topic grouping from the effect of quotas. The results support decay

Policy	Stationary (%)	Category shift (%)	Pollution scan (%)
CARMA	54.636%	68.261%	59.200%
One global topic	54.983%	67.874%	59.200%
No decay	55.108%	59.980%	59.200%
No admission	53.025%	67.850%	55.388%
No quota eviction	54.983%	67.874%	59.200%

Table 5: Average valid-hit rate when one CARMA mechanism is disabled or replaced, at cache capacity 100 entries. These variants are not all mutually orthogonal and are not an additional set of final hypothesis tests.

and admission most clearly, not an independent claim that topic quotas preserve coverage. Consequently, this is a complete component-removal record but only a partial causal ablation: a new experiment would be required to identify the two effects separately.

The recovery-time measurement is inconclusive. In 31 of 40 CARMA transitions and all 40 baseline transitions, the experiment ended before recovery was observed. This is called *right censoring*. A separate time-to-event study would be needed to compare recovery time reliably.

7 Real-system results, MOSS data, and question-pair calibration

The preliminary SQLite and FAISS benchmark replays already-computed synthetic vectors over the same 3,000-request trace. Each policy runs in five new processes. Every run has zero false hits, zero outdated search results, and the same final number of database and vector records. After vector creation, the mean remaining time was 1.804 milliseconds for LRU, 1.724 for LFU, and 1.723 for CARMA. The 95th percentile, meaning 95% of requests were no slower than this value, was 3.511, 3.433, and 3.469 milliseconds. For policy work alone, CARMA’s 95th percentile was 0.354 milliseconds above LRU. In the worst observed run relative to LRU, CARMA’s post-embedding 95th percentile was 1.032 times as large, throughput was 0.997 times as large, and sampled peak physical memory was 1.043 times as large, an increase of 4.8 mebibytes (MiB). Average use of one central-processing-unit core was 96.9% for CARMA and 94.1% for LRU. No graphics processing unit was used. Because detailed rows for embedding and individual requests were not saved, these are summaries per process rather than a full distribution across requests. The next study saves both request and resource measurements.

7.1 Complete request path using the Open Neural Network Exchange format

The saved full-path study measures the complete `gptcache.adapter.adapter.adapt` call. It begins with real text, creates a 768-number ALBERT embedding stored in Open Neural Network Exchange (ONNX) format, searches for similar vectors, reads or writes SQLite and FAISS, runs the eviction and cleanup policy, obtains a recorded answer, and returns it. LRU, LFU, and CARMA receive the same 3,000-request trace with capacity 100 and new storage. Their run order is balanced so that one policy is not always first. The cosine-similarity threshold 0.97 was fixed for reproducible system measurement; it was not selected by a successful final semantic-accuracy test. Five seeds create 15 processes in total, 15,000 requests per policy, 45,000 request records, and 224,804 external measurements of processor and memory use.

The source tag, exact command, run descriptions, file hashes, terminal log, and independent review are listed in the Artifact Map. All values below were recalculated from the saved request, run, and resource records without running the experiment again.

Policy	Mean (ms)	Median (ms)	95th (ms)	99th (ms)
LRU	498.83	497.40	501.45	517.71
LFU	499.46	497.32	505.28	549.59
CARMA	499.68	497.03	506.34	552.01

Table 6: Full-path request time in milliseconds after pooling the 15,000 saved requests within each policy. The 95th and 99th columns are percentiles for individual requests. The five new processes per policy are the independent experimental repeats.

Requests were pooled separately within each policy. Median request time was 497.40 milliseconds for LRU, 497.32 for LFU, and 497.03 for CARMA; the largest difference was 0.37 milliseconds. CARMA’s 95th percentile was 1.010 times the LRU value. Creating the embedding took about 99.3% of average request time. After embedding, average remaining time was 3.28 milliseconds for LRU, 3.17 for LFU, and 3.20 for CARMA. CARMA’s average policy-only work was 0.094 milliseconds above LRU. Average throughput was 2.005, 2.002, and 2.001 requests per second, so CARMA differed from LRU by less than 0.2%. Average processor use was 99.829%, 99.821%, and 99.801%. No graphics processing unit was used. The median of the five process-level peak physical-memory measurements was 398.5, 399.1, and 399.9 MiB. CARMA and LFU each recorded 8,600 hits; LRU recorded 8,200. All 25,400 hits matched the intended concept, stored structure, and recorded response. These statements describe this fixed request trace; they are not estimates of semantic accuracy on unseen questions.

Full-path neural-network measurements

Five fresh processes per policy; 3,000 requests per process; capacity 100

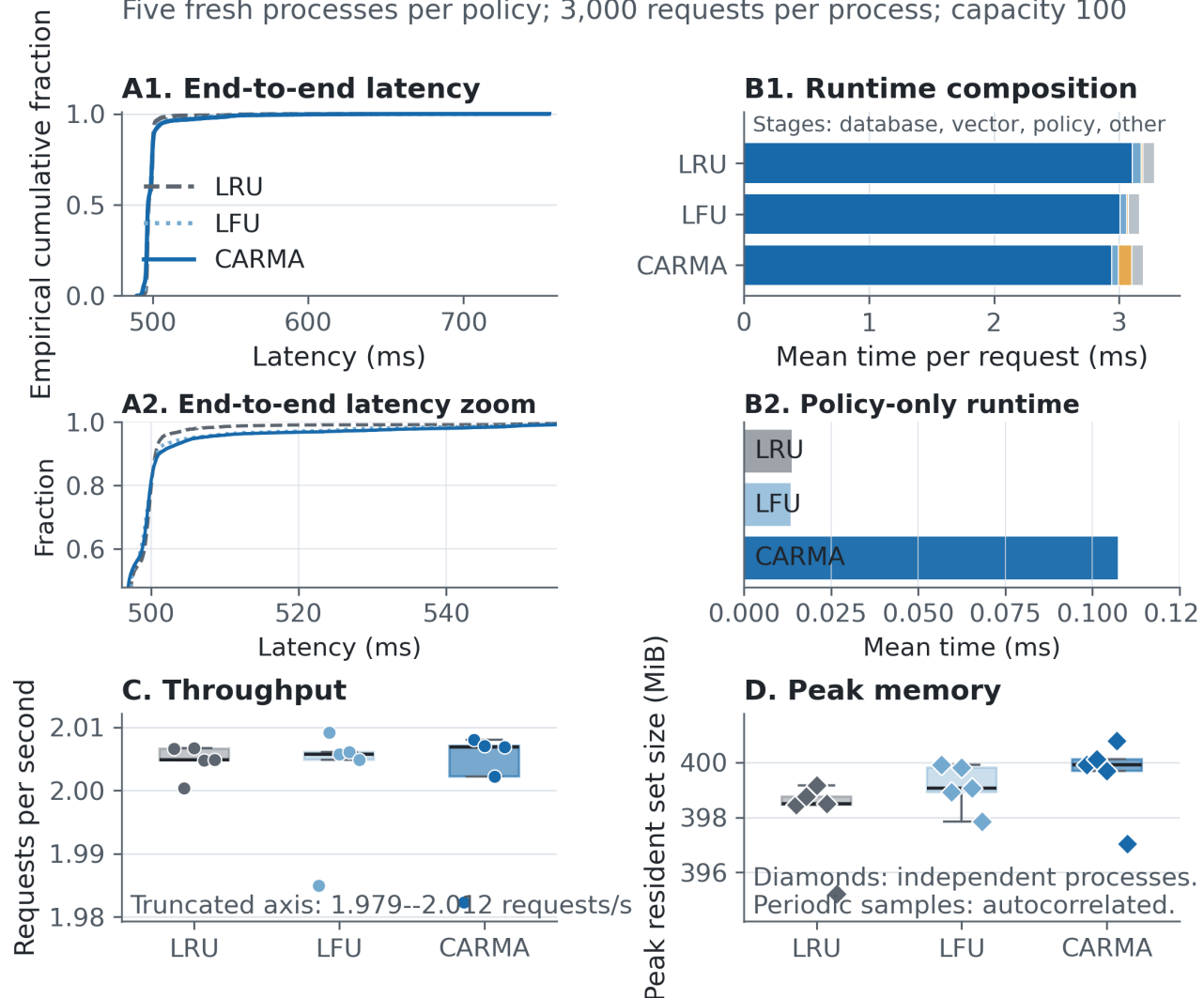


Figure 2: Descriptive full-path ONNX results from saved files. Panel A shows the fraction of requests completed by each time, using 15,000 requests per policy; the lower panel enlarges the range from the median to the 99th percentile. Panel B separates average time after embedding and enlarges the policy-only part. The 495.5–496.5-millisecond average embedding time is not drawn because it is about 99.3% of total average request time. Panel C shows throughput from the five processes on a deliberately narrow scale. Panel D shows peak physical memory for each process. Measurements taken close together in one process are correlated over time, so they are not counted as independent experimental repeats.

7.2 MOSS conversations replayed with exact text keys

The MOSS conversation archive [7] uses the Creative Commons Attribution 4.0 license. The exact source version is fixed by its repository revision and by a 256-bit Secure Hash Algorithm (SHA-256) file fingerprint. A streaming integrity check reads all 301,332 source conversations and 823,634 actual conversation turns. A stable hash then selects the same 2,048-turn sample every time. From that sample, the 200 requests in the longest quarter contain 557,243 tokens across context, prompts, and answers.

This replay uses an LRU cache that requires an exact text key. Every request is unique by design, so all 200 requests must miss the cache. The run delivered all 200 previously recorded MOSS answers and made no live model calls. It also produced zero cache hits and zero false hits. Two runs produced files that were identical byte for byte. This verifies the selected source data and token accounting. It only partly covers the proposed novel-long workload: it does not compare semantic CARMA, LRU, and LFU policies, and it does not measure CARMA’s complete cost on long prompts.

The archive also contained 30,000 rows whose `conversation_id` was the special value -1, and 2,727 conversations whose stated number of turns did not match the rows that were present. Before reporting results, the parser was changed to hash a standard form of rows with negative special identifiers. It also treats consecutive chat keys as the source of truth while keeping both the stated and observed turn counts. The final run description records these data problems and preserves the separate description of the preparation step.

7.3 Duplicate-question calibration and the untouched final test set

This experiment is separate from the comparison of CARMA, LRU, and LFU. It tests whether embedding similarity can classify two questions as duplicates. The calibration set contains 7,729 pairs with no connected question components shared with the final test set: 5,652 pairs are labeled duplicates and 2,077 are labeled non-duplicates.

Twenty similarity thresholds from 0.80 to 0.99 were evaluated. As the threshold became stricter, *precision* generally rose from 92.15% to 98.11%. Precision asks: among pairs predicted to be duplicates, what percentage truly were duplicates? At the same time, *recall* fell from 74.61% to 1.84%. Recall asks: among all true duplicate pairs, what percentage did the method find? The false-positive rate among non-duplicate pairs fell from 17.28% (359/2,077) to 0.096% (2/2,077).

At threshold 0.97, used in the full-path system test, 495 pairs were predicted to be duplicates. Of these, 482 were true duplicates and 13 were not. Precision was therefore $482/495 = 97.37\%$. Recall was $482/5,652 = 8.53\%$: the method found only a small fraction of all true duplicate pairs. The false-positive rate was $13/2,077 = 0.626\%$. Threshold 0.97 was kept so the calibration and system descriptions use the same operating point; it was not chosen using the final test set. At threshold 0.99, 106 pairs were predicted as duplicates: 104 were true duplicates and two were not. Observed precision was 98.11%, recall was 1.84%, and a one-sided 95% Wilson lower confidence bound for precision was 94.46%.

Before looking at the final test set, the protocol required the one-sided 95% lower confidence bound for precision to reach at least 99%. No calibration threshold met that rule. Therefore no threshold was selected, the final test set was not opened, and the planned final-test criteria were not evaluated.

The Quora Question Pairs labels say whether two questions are duplicates. They do not say that one stored answer is factually correct, current, and suitable for both questions. The separate final-test set contains 56,963 pairs, but it was not evaluated. Therefore this report has no final-test semantic metrics and no semantic-accuracy comparison between CARMA and the baseline policies.

Quora Question Pairs calibration trade-off

Held-out not evaluated; 5,652 positive and 2,077 negative calibration pairs

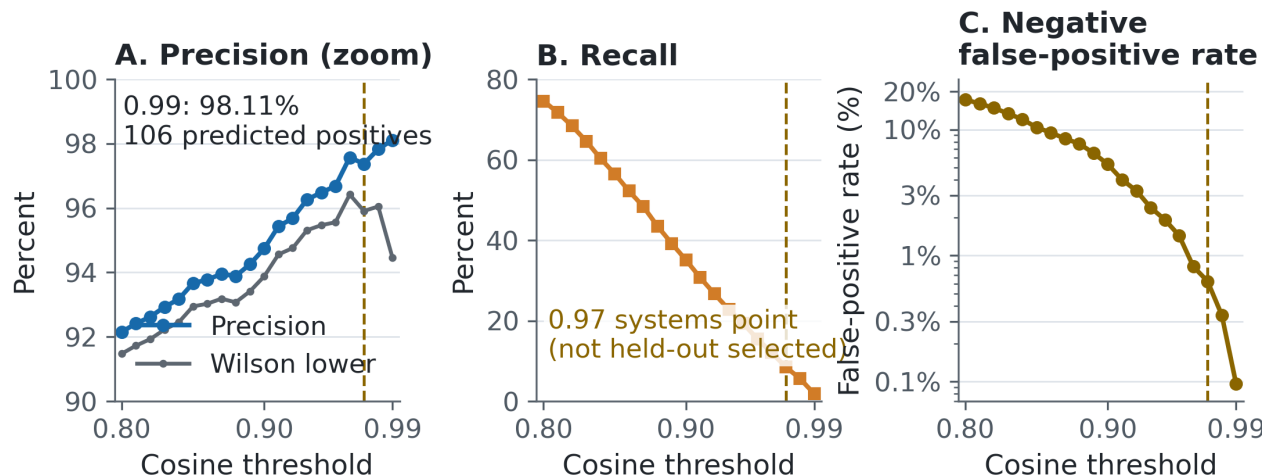


Figure 3: Duplicate-question classification on the Quora Question Pairs calibration set across 20 cosine-similarity thresholds. The precision panel is enlarged to 90–100% and includes the one-sided 95% Wilson lower confidence bound. Recall uses all true duplicate pairs as its denominator; the false-positive rate uses all non-duplicate pairs. The 0.97 line marks the threshold used in the system experiment, not a threshold selected from final test accuracy. At 0.99, precision is based on 106 predicted duplicate pairs. No final-test result is shown because that data was not evaluated.

8 Discussion and limitations

8.1 What the measured results support

The clearest evidence of adaptation is the +4.885-percentage-point result for changing categories, because 80% of demand moves between phases in that workload. The stationary improvement is numerically larger, +7.122 points, but stationary traffic does not test adaptation to change. When individual CARMA components were disabled, removing time decay reduced changing-category valid-hit rate by 8.281 points, and removing admission reduced pollution-scan valid-hit rate by 3.812 points. The experiment cannot separate broad topic clustering from topic quotas because the one-global-topic and no-quota versions produce exactly the same results.

The return phase after pollution shows almost no difference: +0.0067 percentage points, with a confidence interval that includes zero. In the full-path system test, the largest difference in median request time was 0.37 milliseconds. CARMA’s average throughput was less than 0.2% below LRU, and median peak physical memory ranged only from 398.5 to 399.9 MiB across policies. CARMA’s average policy-only work was 0.094 milliseconds above LRU, while embedding creation used about 99.3% of total request time.

8.2 Limits of the experiment and its generalizability

The older synthetic result bundle keeps summary comma-separated-value files and metadata, but not the promised line-by-line JavaScript Object Notation (JSONL) records for requests and process resources. Individual events inside a phase therefore cannot be reconstructed from that bundle alone. The order of policies in synthetic tests was fixed, although each policy had separate state and request timing was turned off. The predeclared large-scale sweep and extra negative and ceiling controls were not run. The report instead includes the saved capacity sweep and four component-removal tests. Preliminary integration timing used synthetic vectors computed in advance; the later ONNX experiment measures the complete request path from text to returned answer.

Synthetic vectors make eviction behavior easier to isolate, but they are not natural language. Quora Question Pairs tests duplicate-question classification, not answer correctness, and its final test set was not evaluated. The MOSS test uses exact text keys and recorded answers; it does not compare semantic cache policies. The ONNX experiment uses macOS 26.5.2 on an arm64 Apple computer with 16 GiB of physical memory, Python 3.12.13, the ONNX CPU execution provider, cache capacity 100, SQLite and FAISS, and five processes per policy. This is

a fixed-host comparison, not a cross-machine performance estimate. Recent-demand and hit scores are not saved across a restart; they begin at zero and are learned again.

9 Conclusion and future work

CARMA adds bounded grouping by meaning, time-decayed demand, candidate admission, topic-based space allocation, and safe cleanup across SQLite and FAISS. In the saved synthetic results, CARMA’s valid-hit rate for changing categories was 4.885 percentage points above the better baseline (95% confidence interval [4.677, 5.088], adjusted $p = 0.0234$), a 7.71% relative improvement. This claim is limited to the controlled synthetic category-shift workload; it is not evidence of cached-answer correctness on unseen natural language. In the return phase after pollution, the difference was only +0.0067 points (95% confidence interval [0, 0.0167], adjusted $p = 1.0$), which does not show a reliable advantage. The full-path experiment reports request-time distributions, where time was spent, throughput, processor use, and memory on one computer; it does not show a latency or throughput improvement for CARMA. The Quora Question Pairs calibration shows the trade-off between precise and broad duplicate detection, but final-test duplicate accuracy and cached-answer correctness remain unknown.

Future work should use a time-to-event method to measure recovery, save adaptive counters in a versioned data format, evaluate the untouched final test set and conversational questions, and repeat the full-path experiment on other computers, cache capacities, and embedding models. The pull request can be merged and its Developer Certificate of Origin check is complete. However, a project maintainer has not reviewed or adopted it, so this report does not claim the optional exceptional-credit condition.

Appendix: Artifact Map

Evidence area	Exact repository locations
Public repository and verified revision	github.com/talya189/gptcache-online-policy , branch <code>feature/gate7-end-to-end</code> , commit <code>bea5c64ebfac5a00632d8c3db046518d668c42a8</code> .
Implementation and tests	<code>gptcache/manager/</code> and <code>tests/</code> .
Benchmark documentation	<code>benchmarks/carma/README.md</code> and <code>docs/project/reproducibility.md</code> .
Environment and continuous integration	<code>Dockerfile.project</code> , <code>requirements-project.lock</code> , <code>requirements-benchmark.lock</code> , and <code>.github/workflows/carma-ci.yml</code> . The exact verification run is GitHub Actions run 33481765384 ; the bounded command is <code>python -m benchmarks.carma --output /tmp/carma-ci</code> .
Synthetic evidence	<code>artifacts/samples/full/runs.csv</code> , <code>aggregate.csv</code> , <code>validation.csv</code> , <code>validation_runs.csv</code> , and <code>metadata.json</code> ; source result commit <code>28129f0b9785232827e741c0e2ff2bb96cc19423</code> .
Public full-path Open Neural Network Exchange evidence	<code>artifacts/gate7-v5-onnx-attempts/attempt-20260831T165407Z-31743/runs.csv</code> , <code>manifest.json</code> , and <code>gate7-adjudication.json</code> . The terminal chain is <code>artifacts/gate7-v5-onnx-attempts/attempt-ledger.jsonl</code> . Source commit <code>f59d64d93a5599149538003078403f1b33e2bab1</code> , tag <code>gate7f-onnx-v5-talya-formal-source</code> .
Full-path raw supplement	The 45,000 request rows and 224,804 resource samples are retained as <code>requests.jsonl</code> and <code>resources.jsonl</code> in the separately submitted archive <code>carma-gate7-v5-raw-evidence.tar.gz</code> ; these large files are deliberately excluded from Git. The archive also contains the public summaries, <code>manifest</code> , <code>adjudication</code> , <code>terminal ledger</code> , and a SHA-256 <code>manifest</code> . Its archive SHA-256 is <code>f29a0b97992511a4f3e9128030ec33cb67a97094f527f2d157f7806c33fb0e76</code> .
Quora Question Pairs calibration	<code>artifacts/samples/qqp/evaluation/calibration_thresholds.csv</code> and <code>result.json</code> .
MOSS dataset replay	<code>artifacts/samples/moss/runs.csv</code> , <code>requests.jsonl</code> , and the retained <code>manifests</code> .
Figures and derived summary	<code>scripts/build_report_descriptive_figures.py</code> , <code>artifacts/samples/analysis/report-descriptive/</code> , and <code>docs/project/chart-map.md</code> . The generator reads retained CSV/JSON only.

The upstream contribution is [GPtCache pull request 696](#). It is open and DCO-passing but had not been reviewed

or adopted by a maintainer at submission time; it is therefore evidence of publication, not the optional adopted-contribution condition.

Exact benchmark commands, environment isolation, and expected outputs are documented in `benchmarks/carma/README.md` and `docs/project/reproducibility.md`; the retained terminal ledger records the full-path invocation.

References

- [1] Bang Fu and Di Feng. “GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings.” *NLP-OSS 2023*, pp. 212–218. [doi:10.18653/v1/2023.nlposs-1.24](https://doi.org/10.18653/v1/2023.nlposs-1.24).
- [2] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu. “SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models.” arXiv:2406.00025v1, 2024. arxiv.org/abs/2406.00025.
- [3] Theodore Johnson and Dennis Shasha. “2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm.” *VLDB 1994*, pp. 439–450. vldb.org/conf/1994/P439.PDF.
- [4] Donghee Lee et al. “LRFU: A Spectrum of Policies that Subsumes the Least Recently Used and Least Frequently Used Policies.” *IEEE Transactions on Computers* 50(12), 2001, pp. 1352–1361. [doi:10.1109/TC.2001.970573](https://doi.org/10.1109/TC.2001.970573).
- [5] Nimrod Megiddo and Dharmendra S. Modha. “ARC: A Self-Tuning, Low Overhead Replacement Cache.” *FAST 2003*. [USENIX proceedings page](#).
- [6] Gil Einziger, Roy Friedman, and Ben Manes. “TinyLFU: A Highly Efficient Cache Admission Policy.” *ACM Transactions on Storage* 13(4), Article 35, 2017. [doi:10.1145/3149371](https://doi.org/10.1145/3149371).
- [7] OpenMOSS Team. “moss-003-sft-data,” revision `42e216d3e3fb331c18d5fa6e7cb4f1c53eef24a4`, licensed under Creative Commons Attribution 4.0. [dataset](#).